SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 35%

Duration: 1 Hour

QUESTIONS: 5 Total Marks:50

Annexure A

Scenario Based Assignment

Code of Conduct:

I P.Revanthsai / 192525451 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Revanthsai

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

iii. Analyze the impact of heuristic accuracy on both algorithms

iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

Answer 1: AI-Powered Medicine Delivery Drone in a Flood Zone

1. Understanding the Scenario and Problem Formulation

The drone has to carry emergency medicine across a flood-affected region where the situation changes minute by minute. Roads and bridges can disappear without warning, rain and wind change the energy and time needed to fly a stretch, and the drone receives only partial real-time information about blocked paths. Because patients are waiting, the route must be found quickly, it must be short in time and risk, and it must be safe for the drone itself. This is a **path-planning problem on a weighted graph in a dynamic, partially known environment**, which is exactly the kind of problem informed search techniques are designed for.

The problem can be formulated in the standard search form as follows:

- **State:** the current location (grid cell or waypoint) of the drone together with its battery level.
- **Initial state:** the base station or depot from where the drone takes off.
- **Goal test:** the drone is at the relief camp, hospital or drop point.
- **Actions:** move to a neighbouring waypoint (north, south, east, west and diagonals) that is not currently reported as blocked.

- **Path cost g(n):** the actual cost from the start to node n. It is not just distance; it is a weighted sum of travel time, wind and rain penalty, terrain difficulty and a risk factor (for example, flying over fast-moving water or damaged power lines).
- **Heuristic h(n):** the straight-line (Euclidean) distance from node n to the goal. It is cheap to compute and never needs a map update.

The straight-line distance is an optimistic estimate because no real route can be shorter than a straight line, so it is **admissible** for distance-based cost. However, once weather and terrain penalties are added to the real cost, the straight line becomes a very loose lower bound, and this fact affects every algorithm discussed below.

i. Behaviour of Greedy Best-First Search (GBFS) in this Scenario

Greedy Best-First Search always expands the node that appears closest to the goal. Its evaluation function is **f(n) = h(n)**. It completely ignores the cost already spent to reach a node. In the drone problem the algorithm keeps a priority queue ordered by straight-line distance to the drop point and always flies toward the waypoint that looks nearest to it.

How it behaves step by step. (1) The drone starts at the depot and puts its neighbours in the queue with their straight-line distances. (2) It chooses the neighbour with the smallest h value and moves there. (3) New neighbours are added and again the smallest h value is chosen. (4) If a path is blocked, the node is removed from consideration and the next best h value is taken. (5) The search ends as soon as the goal is generated. In open terrain this produces an almost straight flight and the answer comes very quickly.

Strengths under uncertainty

- **Speed:** it expands very few nodes, so a first route is produced in milliseconds. This matters when the situation is urgent and the drone has limited onboard computing power.
- **Low memory:** only a small frontier is stored, which suits a small embedded processor on a drone.
- **Simple replanning:** because it needs only h(n), the route can be recomputed instantly when a new blocked-path message arrives.
- **Direction-focused:** it never wastes time exploring away from the goal, so it responds to sudden changes by quickly picking the next most promising node.

Weaknesses under uncertainty

- **Not optimal:** it may select a path that looks short but passes over a flooded valley with strong winds, because costs are ignored. The medicine may arrive later than by a longer but calmer route.
- **Can be trapped:** the straight-line estimate does not know about lakes, flooded rivers or collapsed bridges. The drone may fly toward the goal into a dead end, then have to back up, wasting battery.
- **Not complete in all cases:** in graph search without a visited list it can loop between two nodes that both look close to the goal; even with a visited list it can fail to find the best way around a large obstacle.

- **Blind to risk:** it has no notion of safety cost, so it can lead the drone into dangerous terrain.
- **Fragile to wrong information:** if the real-time update is partial or stale, the greedy choice may be based on wrong data and there is no cost information to correct it.

Illustration. Suppose the drone can go through node B (h = 4 km, but crossing flood water with real cost 12) or node C (h = 6 km, calm terrain with real cost 7). Greedy search prefers B because 4 < 6, although C leads to a cheaper and safer total path.

Node	h(n) (km)	Real cost to goal	Greedy chooses?
B (flooded river route)	4	12	Yes (smallest h)
C (calm hill route)	6	7	No

ii. How A* Handles the Same Situation

A* uses the evaluation function **f(n) = g(n) + h(n)**, where g(n) is the exact cost from the depot to n and h(n) is the estimated cost from n to the goal. The node with the lowest f is expanded first. This means A* considers both how much has already been spent (time, battery and risk) and how much is likely still to be spent.

Balancing cost and heuristic. g(n) keeps the algorithm honest: a node reached through bad weather has a high g value and is discouraged even if it looks close to the goal. h(n) keeps it focused: nodes that point away from the goal receive a high f value and are not explored. When h is **admissible** (never overestimates) A* is guaranteed to return an optimal path; when it is also **consistent** ($h(n) \le c(n, n') + h(n')$) the first time a node is expanded its g value is already the best, so no node needs to be re-opened.

Behaviour in the flood scenario.

- Terrain and weather are added to the edge cost, so a stretch with strong wind and heavy rain costs more than the same distance in calm air. A* will naturally prefer a slightly longer but calmer route.
- When a blocked path is reported, the edge cost is set to infinity (or the edge is removed) and the open list is updated; A* then expands the next best alternatives automatically.
- A risk term can be added to g(n) so that dangerous zones are avoided; this gives the drone safety-aware routes without changing the algorithm.
- A* is **complete** on a finite graph with positive edge costs and **optimal** with an admissible heuristic, so a route is found whenever one exists and it is the best available given the current information.

Worked example. Using the same two options as before, A* computes f for each. Suppose the cost already spent to reach B is 3 and to reach C is 2. Then f(B) = 3 + 4 = 7 at first, but as soon as the drone observes the flooded river the true remaining cost of B becomes 12, so f(B) rises to 15, while f(C) = 2 + 6 = 8 is revised only slightly. A* then switches to C, avoiding the trap, whereas Greedy search would have continued with B.

```
A*(start, goal):
    open <- priority queue ordered by f = g + h; insert start with g = 0
```

```
    closed <- empty set
    while open is not empty:
        n <- node in open with the lowest f(n)
        if n is the goal: return path(n)
        move n to closed
        for each neighbour m of n that is not reported blocked:
            new_g <- g(n) + cost(n, m)        # time + weather + terrain + risk
            if m not in open/closed or new_g < g(m):
                g(m) <- new_g;  f(m) <- new_g + h(m);  parent(m) <- n
                insert/update m in open
    return failure
```

Feature	Greedy Best-First	A*
Evaluation function	f(n) = h(n)	f(n) = g(n) + h(n)
Uses cost so far?	No	Yes
Optimal?	No	Yes, if h is admissible
Complete?	Not always	Yes (finite graph, positive costs)
Speed / nodes expanded	Very fast, few nodes	Slower, more nodes
Memory	Low	Can be high (stores all generated nodes)
Safety / terrain awareness	None	Built into g(n)

iii. Impact of Heuristic Accuracy on Both Algorithms

The heuristic is the only knowledge about the goal that the algorithms use, so its accuracy decides both the speed and the quality of the path.

Effect on Greedy Best-First Search. Greedy search depends entirely on h. If h is accurate and the terrain is open, it is excellent: it goes almost directly to the goal. If h is inaccurate, for example because a river lies between the drone and the goal, the algorithm is misled and may spend a long time exploring the wrong side. Its path quality can become arbitrarily bad, since there is no cost term that can correct a wrong estimate. In the worst case the time and space can reach O(b^m), where m is the maximum depth.

Effect on A*. A* is more tolerant because g(n) corrects errors. The accuracy of h controls how many nodes are expanded:

- **h(n) = 0:** A* becomes Dijkstra / uniform-cost search. It is still optimal but explores in all directions, which is too slow for an urgent mission.
- **h(n) = exact cost h*(n):** A* goes straight to the goal and expands only nodes on the best path. This ideal is unreachable in a changing flood zone.
- **0 < h(n) ≤ h*(n) (admissible, underestimating):** A* remains optimal; the closer h is to h*, the fewer nodes are expanded. Straight-line distance falls here for pure distance, but when weather penalties make real cost much higher it is a weak lower bound and A* expands many nodes.
- **h(n) > h*(n) (overestimating, inadmissible):** A* may be faster but loses its guarantee of optimality and might skip the safest path. Weighted A* ($f = g + w \cdot h$ with $w > 1$) deliberately does this to gain speed at a bounded loss: the path cost is at most w times the optimal cost.

Heuristic quality	Greedy Best-First	A*
Very accurate	Fast and near-optimal	Fast and optimal; very few expansions
Admissible but weak (e.g. straight line only)	Wanders, can be misled by obstacles	Optimal but explores many nodes
Overestimates (inadmissible)	Already unreliable	Faster, but optimality is lost (bounded in weighted A*)
Wrong due to stale or partial updates	Can fly into dead ends; cannot recover cheaply	Corrects through g(n) and re-expansion; degrades gracefully

Improving the heuristic for the drone. Instead of plain straight-line distance, the drone can use a heuristic that includes a weather factor (for example, distance multiplied by the minimum wind penalty over the region), a precomputed map of known flooded zones, and landmark-based (ALT) lower bounds. These keep admissibility but are tighter, so A* becomes both optimal and quick.

iv. Effect of Dynamic Changes (Blocked Paths) on Search Performance

In this scenario the environment is not static. Whenever a road, bridge or air corridor becomes unsafe the graph changes after a plan has been computed. This affects the two algorithms in the following ways.

- **Invalid plans:** a previously computed path may suddenly contain a blocked edge. The drone must detect this and replan; if the block is discovered late (close to the obstacle) the detour is long and costly.
- **Replanning cost:** A* recomputed from scratch after every change is expensive. Greedy search recomputes quickly but gives poorer paths. Incremental algorithms such as **D* Lite**, **LPA*** and **Anytime Repairing A* (ARA*)** reuse earlier search effort and update only the affected part of the search tree, which is much cheaper.
- **Stale heuristic and cost data:** weather changes alter edge costs. Costs measured earlier might not hold, so the g values must be refreshed.
- **Partial observability:** the drone sees updates only for part of the area, so it plans with an incomplete map. A common approach is the **freespace assumption**: unknown cells are assumed passable, and the plan is corrected as new obstacles are observed.
- **Oscillation:** with frequent changes, the drone may flip between two routes and waste battery. A small hysteresis (change route only if the improvement is above a threshold) avoids this.
- **Real-time deadlines:** every replanning step must finish within a fixed time. Anytime algorithms return a reasonable path quickly and improve it while time allows.

Dynamic event	Greedy Best-First	A* (from scratch)	D* Lite / Anytime A*
Single edge blocked	Quick local re-choice; may pick a poor detour	Full re-search; optimal but slower	Repairs only affected nodes; fast and optimal
Many edges blocked	May dead-end repeatedly	Heavy recomputation	Moderate cost, still correct

Dynamic event	Greedy Best-First	A* (from scratch)	D* Lite / Anytime A*
Costs change (wind, rain)	Ignores costs	Needs recomputing g values	Updates inconsistent nodes only

v. Recommendation and Justification

Recommended algorithm: A* (in practice a weighted / anytime version with incremental replanning such as D* Lite), using a tight admissible heuristic and a risk-aware cost function.

Real-time constraints. Plain A* can be slow on a large map, but weighted A* or Anytime Repairing A* returns a good route within milliseconds and keeps improving it while the drone flies. D* Lite repairs the route when new blocked paths are reported, so replanning stays within the time limit. Greedy search is faster, but the time saved is small compared with the delay caused by one wrong turn.

Optimality. A* gives the lowest-cost route (time plus energy plus risk) for the information currently known. In a medical emergency, arriving by the shortest-time reliable path matters more than the first path that is found, and a bounded-suboptimal weighted version guarantees the result is within a known factor of the best.

Safety. The cost function can include risk factors, so the drone avoids high-wind corridors, unstable bridges and areas near power lines. Battery level can be part of the state so that the planner never selects a path that cannot be completed, and a safe landing or return point can always be reserved. Greedy search has no such mechanism.

Handling uncertainty. Because A* variants are complete, the drone will still find a way when the direct route is blocked, whereas greedy search might get stuck. When data are partial, the drone applies the freespace assumption and replans as new sensor information arrives.

Practical hybrid. A reasonable design is: use Greedy Best-First as an emergency fallback if the computing budget is nearly exhausted (for instance, a very short time left to make a decision), and otherwise run Anytime A*/D* Lite. This combines the speed of greedy search with the reliability and safety of A*.

Criterion	Greedy Best-First	A* (weighted / D* Lite)
Speed of first answer	Excellent	Good (anytime)
Route quality (optimality)	Poor, unbounded	Optimal / bounded sub-optimal
Adaptation to blocked paths	Quick but weak	Strong with incremental repair
Safety and risk handling	None	Included in cost function
Overall suitability	Backup only	Recommended

Conclusion. For an emergency drone in a flood region, delay and risk are both critical. Greedy Best-First Search is attractive for speed but unreliable under uncertainty. A* with an admissible, weather-aware heuristic, enhanced by incremental replanning, balances cost and heuristic, guarantees good and safe routes, and adapts to changing conditions, so it is the most suitable choice.

2. Scenario: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

Answer 2: Smart City Traffic Signal Optimization

1. Understanding the Scenario

A smart city wants to control the timing of traffic signals at hundreds of intersections so that vehicles spend less time waiting, burn less fuel and cause less congestion. The signals must be adjusted continuously as traffic flow changes with the time of day, accidents, events and weather. The number of possible timing combinations is enormous, the traffic pattern keeps changing, and simple improvement methods can become stuck in solutions that are good locally but poor for the city as a whole. This is a **large-scale, dynamic, multi-objective optimization problem** and not a path-finding problem, because there is no single goal state: we want the best configuration, not a route.

2. Formulation as an Optimization Problem

- **State (solution):** a complete signal-timing plan, represented as a vector X = (c, g1, g2, …, gk, o1, o2, …, ok) for each intersection i, where c is the cycle length, g is the green time of each phase (split) and o is the offset that decides how well neighbouring signals are coordinated into a 'green wave'.
- **Search space:** if each of 300 intersections has only 10 possible green splits and 10 possible offsets, the number of plans is 100^300 = 10^600. Exhaustive enumeration is impossible.
- **Objective function (to minimize):** F(X) = w1·W(X) + w2·Fuel(X) + w3·Cong(X), where W is total vehicle waiting time, Fuel is the fuel used (which grows with stops and idling) and Cong is a congestion measure such as average queue length or the number of saturated links. The weights w1, w2, w3 reflect the city's priorities. Equivalently, we can maximize a utility U(X) = −F(X).

- **Constraints:** minimum and maximum green time per phase, a minimum pedestrian crossing time, a limit on the cycle length, safety clearance (yellow and all-red) intervals, priority for emergency vehicles and public transport.
- **Evaluation:** F(X) is computed with a traffic simulator or from live sensor data (loop detectors, cameras, GPS), so each evaluation of a candidate has a real cost.
- **Dynamic nature:** the best X at 9 a.m. differs from the best X at 2 p.m.; the problem must be re-solved repeatedly, so the optimizer should be able to start from the current solution.

Why traditional (systematic) search is inefficient.

- **Exponential search space:** BFS, DFS and uniform-cost search explore states one by one, and the branching factor is huge. They cannot finish in the time between two changes in traffic.
- **No clear goal state or path:** classical search looks for a path from a start to a goal. Here only the final configuration matters, and the quality is a number, so storing paths is wasteful.
- **No good admissible heuristic:** A* needs an estimate of the remaining cost. For signal timing it is very difficult to give a trustworthy estimate because intersections interact.
- **Memory:** keeping a frontier of billions of states is not feasible.
- **Dynamic environment:** by the time a systematic search finishes, the traffic has changed and the solution is outdated.

Therefore **local search and population-based metaheuristics** are used: they keep only the current solution (or a small set), need little memory and give a good answer at any time, which suits this scenario.

i. How Hill Climbing Works and its Limitations

Hill Climbing starts with a complete signal plan (for instance the plan currently running) and repeatedly moves to a better neighbouring plan. A neighbour is obtained by a small change, such as increasing the green time of one phase by 2 seconds, shifting one offset, or swapping the cycle length. The change is accepted if it reduces the cost F(X); otherwise it is rejected. The algorithm stops when no neighbour is better.

```
HillClimbing(initial_plan):
    current <- initial_plan
    loop:
        neighbours <- small modifications of current (green split, offset, cycle)
        best <- neighbour with the lowest F value
        if F(best) >= F(current): return current      # no better neighbour
        current <- best
```

Variants. Steepest-ascent hill climbing checks all neighbours and takes the best; first-choice (stochastic) hill climbing takes the first improving neighbour, which is cheaper when there are many neighbours; random-restart hill climbing repeats the search from several random starting plans and keeps the best result.

Advantages. It is simple, uses very little memory, is fast, can start from the current running plan, and gives a better plan after every step, so it can be stopped at any time.

Limitations in this scenario.

- It gets stuck at **local optima** because it never accepts a worse move.
- It cannot cross **plateaus** where many neighbours have the same value.
- It struggles with **ridges** where several variables must change together.
- It is **greedy and short-sighted**: improving one intersection may worsen another, and the algorithm only sees the immediate effect.
- The result depends heavily on the **starting plan**; two different starts lead to two different answers.
- In a noisy, dynamic environment a measured improvement may only be random fluctuation in traffic.

ii. Local Maxima, Plateaus and Ridges: Real-World Interpretation

Problem	Meaning in the algorithm	Real-world traffic interpretation
Local maximum (optimum)	A plan better than all its neighbours but worse than the global best; every single small move makes it worse	A main road has a perfect green wave, but the side roads are starved of green time and long queues build up there. Any small change to the main road's timing increases its delay, so the algorithm stops, although a different overall arrangement would be better for the whole city.
Plateau (flat area)	A region where neighbouring plans all have nearly the same objective value, so there is no direction to follow	During night or off-peak hours, changing green time by a few seconds does not change total waiting time at all, because there are few vehicles. The algorithm wanders without knowing where to go. A shoulder is a plateau with a way up at the end, for example a junction whose benefit only appears after a larger change.
Ridge	A narrow path of improvement that is not aligned with the single-variable moves; the neighbours reachable by one-step changes all look worse even though a combined change is better	To create a green wave, the offset of intersection A and of intersection B must be changed together. Shifting only A breaks the existing coordination and makes things worse, and shifting only B does the same; only the combined change improves traffic. Single-parameter hill climbing cannot see this.
Noise / shifting landscape	The objective value changes even without changing the solution	An accident, a school closing or a sudden rain makes yesterday's best plan poor. The landscape itself moves.

These difficulties show that a method must be able to **accept temporary worsening** (to escape local maxima), **move over flat regions** (plateaus) and **change several variables together** (ridges).

iii. How Simulated Annealing or Genetic Algorithms Overcome These Limitations

A. Simulated Annealing (SA). SA is inspired by slowly cooling metal so that it reaches a low-energy, stable state. It behaves like hill climbing but sometimes accepts a worse neighbour. If

the neighbour has cost change ΔE = F(new) − F(current): when ΔE < 0 (better) it is always accepted; when ΔE > 0 (worse) it is accepted with probability **P = e^(−ΔE / T)**, where T is the temperature. At the start T is high, so the algorithm explores widely; as T is decreased gradually (cooling schedule, for example T ← 0.95·T) uphill moves become rare and the search settles into a good solution.

```
SimulatedAnnealing(initial_plan, T0, alpha):
    current <- initial_plan;  T <- T0
    while T > T_min:
        next <- random neighbour of current
        dE <- F(next) - F(current)
        if dE < 0 or random(0,1) < exp(-dE / T): current <- next
        T <- alpha * T                      # cooling
    return best plan found
```

How SA overcomes the limitations: **local maxima** are escaped because worse moves are sometimes accepted; **plateaus** are crossed since random moves are accepted with some probability; **ridges** are handled because the temporary downhill moves let the search follow the ridge. It is still simple and needs only one solution in memory, and a plan can be refreshed by 'reheating' when the traffic pattern changes.

B. Genetic Algorithms (GA). GA is inspired by natural selection and works with a **population** of candidate signal plans.

- **Chromosome (encoding):** a plan encoded as a string of numbers, e.g. [cycle, g1, g2, o1, …] for every intersection.
- **Fitness:** the inverse of the objective F, evaluated by a traffic simulator (shorter waiting, less fuel and less congestion give higher fitness).
- **Selection:** better plans are more likely to be chosen as parents (tournament or roulette-wheel selection).
- **Crossover:** the plans of two parents are combined, for example by taking the timings of one district from parent 1 and another district from parent 2, so groups of related variables (a whole corridor) are transferred together, which handles ridges.
- **Mutation:** a few random changes in green times or offsets maintain diversity and help leave local optima.
- **Elitism:** the best plans are copied unchanged into the next generation so quality never drops.

GA explores many regions of the search space in parallel, so it is much less likely to be trapped in a single local maximum. It works well with a black-box simulator, handles several objectives through the weighted fitness (or Pareto methods such as NSGA-II), and is easy to run in parallel on many processors. Its disadvantages are the number of fitness evaluations needed and the need to tune parameters such as population size, crossover rate and mutation rate.

Feature	Hill Climbing	Simulated Annealing	Genetic Algorithm
Solutions kept	1	1	Population (e.g. 50–200)
Escapes local maxima?	No	Yes (probabilistic uphill)	Yes (diversity, mutation, crossover)
Handles plateaus / ridges	Poor	Fair	Good

Feature	Hill Climbing	Simulated Annealing	Genetic Algorithm
Memory	Very low	Very low	Moderate
Speed per iteration	Fast	Fast	Slower (many evaluations)
Parallelism	Limited	Limited	Excellent
Global optimum guarantee	No	Only with very slow cooling	No, but good in practice

iv. Recommended Approach and Justification

Recommendation: a hybrid (memetic) approach, with a Genetic Algorithm for global exploration and Simulated Annealing or hill climbing for local refinement, run in a rolling-horizon manner.

Scalability. For hundreds of intersections, GA can be parallelised over a cluster, and the city can be divided into zones or corridors (decomposition) so that each sub-problem has a manageable size. Simulated annealing and hill climbing need little memory and scale linearly with the number of variables. Plain hill climbing would be the fastest but is unreliable, and an exhaustive search is impossible.

Adaptability. Traffic changes continuously. GA can restart from the previous population (warm start) when conditions change, keeping the good parts of earlier solutions while adapting to new data. SA can be reheated to a moderate temperature to re-optimise. Both can run at any time and give an acceptable plan immediately (anytime behaviour), which is important for a live system.

Solution quality. The global search of the GA avoids the local maxima of the main-road-versus-side-road type, and the local refinement polishes the best plans in a few seconds; this combination gives better results than either part alone.

Practical operation.

- Run the GA every 15–30 minutes (or when the traffic pattern changes significantly) to select a good set of timing plans for each zone.
- Between runs, use hill climbing or SA with small moves and live detector data to fine-tune the active plan.
- Always keep safety constraints (minimum green, pedestrian time, emergency priority) as hard limits that every candidate must satisfy.
- Test the candidate plan in a simulator before applying it to real signals, and fall back to the last safe plan if the new one performs worse.

Conclusion. Traditional search cannot cope with the enormous, changing search space of the traffic-signal problem. Hill climbing is simple and fast but is trapped by local maxima, plateaus and ridges. Simulated Annealing and Genetic Algorithms overcome these limits, and a hybrid of GA (global search) with SA or hill climbing (local refinement) offers the best balance of scalability, adaptability and solution quality for a smart-city traffic system.

3. Scenario: An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search
ii. Analyze the challenges of partial observability and dynamic environments
iii. Describe how the rover builds and updates its internal model
iv. Compare online search with uninformed and informed search strategies
v. Justify the most suitable approach considering uncertainty, adaptability, and safety

Answer 3: Autonomous Mars Rover and Online Search

1. Understanding the Scenario

An autonomous rover on Mars must explore terrain that no one has mapped in detail. It has to reach sampling sites, avoid craters, rocks and soft sand, and make decisions using only the small area it can sense with its cameras and range sensors. Radio signals take from about 3 to 22 minutes to travel one way between Earth and Mars, so operators cannot steer the rover in real time. The rover therefore has to think and act on its own while learning the map as it moves. This is a classical **online search (exploration) problem in a partially observable, dynamic and unknown environment**.

In an **offline search** the agent first computes a complete plan using a full model of the world (states, actions and costs) and then executes it without further thought. In an **online search** the agent interleaves computation and action: it takes an action, observes the result, and only then decides on the next action.

i. Why an Online Search Agent is Required Instead of Offline Search

- **The map is unknown.** Offline search such as A* or BFS needs the complete state space, the transition function (RESULT(s, a)) and the step costs in advance. The rover does not know what lies beyond its sensor range, so it cannot construct the full search tree.
- **Knowledge arrives while acting.** The rover discovers craters, boulders and slopes only when it gets near them. The best plan therefore changes continuously, and a plan computed at the beginning would quickly become invalid.
- **Exploration is part of the task.** The rover must itself find out what actions do and what the terrain looks like, which is the definition of an exploration problem. Offline search has no way of gathering new information.
- **Communication delay and limited bandwidth.** Because of the delay, Earth cannot handle each decision; the rover must decide locally and immediately.

- **Actions may be irreversible.** Driving into a deep crater or sand trap can end the mission. An online agent can be cautious, taking one step at a time and verifying the result, whereas an offline plan would commit to a long sequence of actions based on assumptions.
- **Limited onboard computing and energy.** The rover cannot afford to compute a full optimal solution of a huge space; it computes a little, acts, and uses the new observation.

Online search agents are evaluated using the **competitive ratio**: the cost of the path actually travelled divided by the cost of the optimal path the agent would take if it knew the whole map in advance. In an environment with irreversible actions (dead ends), no algorithm can guarantee a bounded competitive ratio, which is why the rover must emphasise safety.

ii. Challenges of Partial Observability and Dynamic Environments

Partial observability. The rover's sensors give only a limited, noisy view: the field of view is narrow, shadows hide holes, dust reduces visibility and depth estimation is uncertain beyond some distance. The rover therefore does not know its exact state; it holds a **belief state** (a set or probability distribution of states it might be in).

Dynamic environment. The terrain and conditions change: sand dunes shift, dust storms alter visibility and solar power, wheels slip so a movement does not reach the intended position, temperature changes the behaviour of the equipment, and rocks may be dislodged by the rover itself.

Challenge	Description	Effect on decisions
Limited sensing range	Only a small area around the rover is observed	Must plan with an incomplete map; may discover a hazard too late
Sensor noise and ambiguity	A shadow may be a crater; a flat-looking surface may be soft sand	False positives waste time; false negatives risk loss of the rover
Localisation uncertainty	Wheel slip and drift mean the position estimate is imperfect	Wrong position corrupts the map (needs SLAM and correction)
Unpredictable action outcomes	A drive command can result in slipping or being stuck	Need to verify results and replan; actions have probabilistic outcomes
Changing environment	Dust, wind and shifting sand change terrain and visibility	Old map data may become stale; periodic re-sensing required
Irreversible actions / dead ends	Falling into a crater or getting stuck	Requires conservative, safety-first policy
Limited energy, compute and time	Battery, solar power and processor are restricted	Cannot afford wide exploration or heavy computation
Exploration vs exploitation	Explore to learn more or head directly to the sample site	Needs a balance between information gain and progress

Because of these challenges the rover has to maintain and update a model of the world, make decisions that are robust to uncertainty, and prefer actions whose results it can observe and undo.

iii. How the Rover Builds and Updates Its Internal Model

The rover keeps an internal representation of the world that grows with every observation. The usual structures are:

- **Occupancy grid / cost map:** the terrain is divided into cells, each storing the probability that it is free, occupied (rock, wall of a crater) or unknown, and a traversability cost (slope, roughness, softness). Unknown cells are initially treated cautiously.
- **Transition (result) table:** like the online-search agent in the textbook, the rover stores result[s, a] = s′ for every action it has tried, so that it can plan using what it has already learned.
- **Visited and untried-action lists:** for each state, the actions not yet tried and the states to which it can back up.
- **Heuristic estimates H(s):** an estimate of the cost to reach the goal from each state, updated as experience shows the estimates were too low or too high.
- **Pose estimate:** the rover's position and orientation computed by combining wheel odometry, an inertial sensor and visual landmarks (visual odometry or SLAM, simultaneous localisation and mapping).

Update cycle (sense – update – plan – act).

```
loop:
    percept <- sense()                          # cameras, lidar, slope, wheel slip
    update pose estimate (odometry + landmarks)
    for each cell in the sensor range:
        update P(cell is hazard) with Bayes rule   # occupancy grid
        update traversability cost
    record result[prev_state, action] = current_state
    plan next action on the current model (local replanning)
    execute the first action only, then repeat
```

Bayesian update. If a cell has prior probability p of being an obstacle and a sensor reading z arrives, the new belief is P(obstacle | z) = P(z | obstacle)·p / P(z). Repeated consistent readings increase confidence, while contradicting ones lower it. This allows the rover to be robust to noise.

Online DFS agent (textbook algorithm). The rover chooses an untried action in the current state; if all actions are tried, it backtracks to the previous state in its record. This guarantees that every reachable cell is eventually visited in a safe, reversible environment. **LRTA*** (Learning Real-Time A*) instead stores H(s) for each visited state and updates it using H(s) = min over actions of [c(s, a, s′) + H(s′)], then moves to the apparently best neighbour; over repeated visits the estimates become more accurate, so the rover learns to avoid dead ends and loops.

Replanning. After each model update the rover re-evaluates its route, often using incremental planners such as **D* Lite**, which repairs the previous plan when new cells are discovered instead of planning from scratch, making real-time updates efficient.

iv. Comparison of Online Search with Uninformed and Informed Search Strategies

Uninformed (blind) strategies such as BFS, DFS, uniform-cost and iterative deepening use no knowledge about the goal and, in their standard form, assume the whole state space and the

results of actions are known. **Informed strategies** such as greedy best-first and A* use a heuristic to focus the search, but they are also offline: they expand nodes in a model before acting. **Online strategies** (online DFS, LRTA*, D* Lite) expand the node where the agent actually is and learn the model while moving.

Criterion	**Uninformed (BFS / DFS / UCS)**	**Informed (Greedy / A*)**	**Online search (online DFS, LRTA*, D* Lite)**
Needs complete map in advance?	Yes	Yes	No, learns as it moves
Plans before acting?	Yes, whole path	Yes, whole path	Plans a step, acts, and replans
Uses heuristic?	No	Yes (fixed)	Yes, can be learned and updated
Handles unknown / changing terrain	Poorly	Poorly (plan may be invalid)	Well
Safety under uncertainty	No consideration	Only through given costs	Can be conservative; checks outcomes each step
Backtracking	Computational (in model)	Computational (in model)	Physical; real movement, must be reversible
Memory and computation	High (large trees)	Moderate to high	Low per step; stores explored model only
Optimality	BFS/UCS optimal for known map	A* optimal with admissible h	Not guaranteed; measured by competitive ratio
Typical use	Small, fully known problems	Route planning with known maps	Robot exploration, unknown or changing environments

Key points of the comparison. The main difference is the position of the agent: in an offline search the agent explores a 'mental' tree and can discard bad branches at no cost, whereas an online agent must physically move to explore, so wrong steps cost energy and time and can be dangerous. Informed offline search is extremely efficient when a good map exists, but it is of limited use when the map is mostly unknown. Online search combines ideas from both: it uses heuristics to choose promising directions and uninformed exploration (such as depth-first with backtracking) to guarantee that unexplored areas are eventually covered.

v. Most Suitable Approach: Justification

Recommended approach: an online, heuristic search agent such as LRTA* or D* Lite, supported by an occupancy-grid map, Bayesian updating of hazards, and a conservative, safety-aware cost function with local replanning.

- **Uncertainty:** by sensing and updating a probabilistic map after every step, the rover works with the best current belief rather than with an assumed perfect model. Cells with high hazard probability receive large costs or are forbidden, and unknown cells are treated cautiously.
- **Adaptability:** D* Lite repairs the plan whenever new obstacles are seen, and LRTA* improves its heuristic estimates with experience. The rover therefore adapts to new terrain

information, wheel slip and changing environmental conditions without computing everything again.

- **Safety:** the cost function includes slope, roughness and distance to hazards; the planner keeps a safety margin around craters and rocks, avoids irreversible moves when alternatives exist, only drives into regions that have been seen, and always keeps a path back to a known safe state. Energy and thermal limits are included as constraints. If the confidence is low, the rover stops, re-senses or asks Earth for guidance.
- **Efficiency:** computation per step is small, so it fits the rover's limited processor; the rover moves toward the sample site guided by the heuristic while exploring only as necessary.
- **Completeness of exploration:** online DFS-style backtracking in safe, reversible regions ensures that, if the goal can be reached, the rover will eventually find it.

Conclusion. Offline search is impossible when the map is unknown, and uninformed or informed offline methods are not suitable on their own. An online search agent that senses, updates its internal model, replans incrementally and prioritises safety is the best fit for Mars exploration, as it can handle partial observability and dynamic conditions while keeping the rover and the mission safe.

4. Scenario: A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation
ii. Explain how backtracking search works in this scenario
iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency
iv. Analyze real-world challenges and justify the best strategy for scalability

Answer 4: University Exam Timetable as a Constraint Satisfaction Problem

1. Understanding the Scenario

The university needs a timetable that assigns every exam to a day, a time slot and an exam hall so that all rules are respected: no student has two exams at the same time, the number and capacity of halls is limited, some subjects must be examined before others, faculty (invigilators and examiners) are not always available, and students with special accommodations (extra time, a separate room, a ground-floor hall) are supported. As the number of courses and students grows, the number of possible timetables increases explosively. The problem is naturally expressed as a **Constraint Satisfaction Problem (CSP)**, in which we search for an assignment of values to variables that satisfies every constraint.

2. Formulating the Problem as a CSP

A CSP is defined by a triple **(X, D, C)**: a set of variables X, a domain D of possible values for each variable, and a set of constraints C that specify which combinations of values are allowed. A solution is a **complete and consistent assignment**.

i. Variables, Domains and Constraints

Variables (X). One variable for each exam (course) Ci: X = {C1, C2, …, Cn}. For example: C1 = Mathematics, C2 = Physics, C3 = Artificial Intelligence, C4 = Databases, C5 = Operating Systems. If the exam hall is also chosen by the solver, a second variable H(Ci) (hall of exam Ci) is added to each course.

Domains (D). The domain of each Ci is the set of all possible time slots (day, session) in the exam period, for example 10 days × 2 sessions = 20 slots: Di = {D1-AM, D1-PM, D2-AM, …, D10-PM}. The domain of the hall variable H(Ci) is the set of halls {Hall A, Hall B, …} with enough capacity for the exam. Initial domains may be reduced by unary constraints (below).

Constraints (C).

Requirement	Type	Formal constraint	Explanation
No student has overlapping exams	Binary	If courses Ci and Cj share at least one student then Slot(Ci) ≠ Slot(Cj)	Edges in the constraint graph are created between all pairs of courses with common students (a conflict graph); this is the main constraint and makes the problem a graph-colouring problem.
Limited number of exam halls	Global (resource)	For every slot s: number of exams with Slot = s ≤ number of halls; and total students in that slot ≤ total hall capacity; each hall holds at most one exam in a slot	Prevents more exams running at the same time than the university can host. Equivalent to an 'at most k' (cardinality) constraint.
Some subjects before others (precedence)	Binary	Slot(Ci) < Slot(Cj) when Ci must be before Cj	Prerequisite or syllabus ordering, for example practical exams before theory exams.
Faculty availability	Unary	Slot(Ci) ∈ Available(faculty of Ci)	Removes slots in which the examiner or invigilator is on leave or has another duty, directly shrinking the domain.
Special accommodations	Unary / binary	H(Ci) ∈ AccessibleHalls for such students; extra duration means the following slot must be free for them; separate room needed	Ensures students needing extra time or a ground-floor room get appropriate halls and non-overlapping slots.
Hall capacity	Binary (Ci, H(Ci))	Students(Ci) ≤ Capacity(H(Ci))	A hall must be large enough for the course.
Optional preference limits (soft)	Soft / preference	Avoid more than 2 exams for a student on the same day; spread exams evenly	Not required for feasibility but desirable; handled as a cost to be minimised.

Constraint graph example. Suppose five courses and the following common-student pairs: C1–C2, C1–C3, C2–C3, C3–C4, C4–C5. The constraint graph has an edge for each pair, meaning those two exams must be in different slots. Let the domain of each course be {S1, S2, S3}, and suppose there are 2 halls so at most 2 exams per slot, and C1 must be before C4.

ii. Backtracking Search in this Scenario

Backtracking search is a depth-first search that assigns one variable at a time and checks the constraints after each assignment. If the current partial assignment violates a constraint, the algorithm immediately abandons it (backtracks) and tries another value or goes back to the previous variable. This is better than generating all complete assignments and testing them, because a violation is detected as early as possible and the whole subtree below it is skipped.

```
Backtrack(assignment, csp):
    if assignment is complete: return assignment
    var <- SelectUnassignedVariable(csp)           # e.g. MRV + degree heuristic
    for value in OrderDomainValues(var, assignment): # e.g. least-constraining value
        if value is consistent with assignment:
            add {var = value} to assignment
            result <- Backtrack(assignment, csp)
            if result != failure: return result
```

```
            remove {var = value} from assignment      # undo and try next value
    return failure
```

Trace for the example (domains {S1, S2, S3}, max 2 exams per slot, C1 before C4):

- Assign C1 = S1 (consistent).
- Assign C2 = S1? It conflicts with C1 (common students), so try C2 = S2 (consistent).
- Assign C3: S1 conflicts with C1, S2 conflicts with C2, so C3 = S3 (consistent).
- Assign C4: it conflicts with C3 (S3) and must come after C1 (S1), so it can take S2 (consistent; hall limit is 2 exams, so S2 holds C2 and C4, which is allowed).
- Assign C5: it conflicts with C4 (S2), so S1 or S3 is possible; choose S1. All variables are assigned and all constraints hold, so a solution is found: C1 = S1, C5 = S1, C2 = S2, C4 = S2, C3 = S3.

If at some stage a variable had no legal value left (for example if the domain had only two slots, S1 and S2, C3 would have no possible value), the algorithm would backtrack to C2 or C1 and change their values. In a large timetable, naive backtracking can still waste time exploring many hopeless branches, which is why heuristics and constraint propagation are added.

Heuristics to improve backtracking.

- **Minimum Remaining Values (MRV):** pick the variable with the fewest legal values left (for example, an exam whose faculty is available only in two slots). This detects failure early (a 'fail-first' strategy).
- **Degree heuristic:** among tied variables choose the one involved in the most constraints (a course shared by many students). It reduces future branching.
- **Least Constraining Value (LCV):** try first the slot that rules out the fewest options for the neighbouring courses, keeping flexibility for later assignments.

iii. Constraint Propagation (Forward Checking) and Efficiency

Plain backtracking discovers a problem only when it reaches the variable that has no value. **Constraint propagation** uses the constraints to prune domains in advance, so dead ends are recognised earlier.

Forward checking. Whenever a variable X is assigned a value, forward checking looks at every unassigned variable Y connected to X by a constraint and deletes from Y's domain the values that are inconsistent with the new assignment. If any domain becomes empty, the algorithm backtracks at once.

Step	Assignment	Domains after forward checking (C1..C5)	Remark
Start	none	C1 {S1,S2,S3}, C2 {S1,S2,S3}, C3 {S1,S2,S3}, C4 {S1,S2,S3}, C5 {S1,S2,S3}	Initial
1	C1 = S1	C2 {S2,S3}, C3 {S2,S3}, C4 {S2,S3} (after C1), C5 {S1,S2,S3}	S1 removed from neighbours C2, C3 and precedence prunes C4
2	C2 = S2	C3 {S3}, C4 {S2,S3}, C5 {S1,S2,S3}	S2 removed from C3, which is a neighbour of C2

Step	Assignment	Domains after forward checking (C1..C5)	Remark
3	C3 = S3	C4 {S2}, C5 {S1,S2,S3}	S3 removed from C4
4	C4 = S2	C5 {S1,S3}	S2 removed from C5
5	C5 = S1	all assigned	Solution without any backtracking

Benefits. (1) Early failure detection saves exploring entire subtrees. (2) Together with MRV, the domains that forward checking reduces show which variable is most constrained, so the heuristic becomes more accurate. (3) The number of nodes searched falls dramatically, especially when the constraint graph is sparse or heavily constrained.

Stronger propagation. Forward checking does not detect all inconsistencies (it does not check constraints between two unassigned variables). **Arc consistency (AC-3)** enforces that for every value of variable X there is some compatible value in each neighbour Y; the **MAC** (Maintaining Arc Consistency) algorithm runs AC-3 after every assignment and prunes more. Its cost per node is higher, but it often reduces total search time by orders of magnitude on large timetabling problems. Domain reductions from unary constraints (faculty availability, accessible halls) are applied before the search starts, which is the cheapest form of propagation (node consistency).

iv. Real-World Challenges and the Best Strategy for Scalability

Challenges.

- **Problem size:** thousands of courses and tens of thousands of students create a dense conflict graph and a huge search space; CSP solving is NP-complete in general.
- **Over-constrained problems:** there may be no schedule satisfying all requirements. Then we must decide which preferences to relax, which requires soft constraints and optimisation.
- **Soft constraints and fairness:** avoiding three exams in a day, giving gaps between exams, and keeping the load fair among students.
- **Dynamic changes:** late course registrations, a hall becoming unavailable or an examiner falling ill require a quick re-schedule with minimal change to the published timetable.
- **Diverse accommodations and policies:** many special cases make the constraints complex and sometimes conflicting.
- **Data quality and explainability:** incorrect enrolment data cause conflicts; administrators need to understand why a timetable was chosen.
- **Time limits:** the timetable must be produced in hours or a few days, and the user may need several alternatives.

Recommended strategy. No single technique is best at every scale. The most effective approach for a real university is a **hybrid CSP strategy**:

- **Pre-processing:** apply node consistency using unary constraints (faculty and accommodation), and build the conflict graph from enrolment data.

- **Core solver:** backtracking search with MRV + degree heuristic for variable ordering, LCV for value ordering, and forward checking or MAC for propagation. This gives a feasible timetable quickly and is complete, so it can prove that no solution exists.
- **Decomposition:** split the problem into independent or loosely connected parts (by faculty, department or week), solve them separately and combine them, or use cutset conditioning on the constraint graph to reduce complexity.
- **Local search for repair and optimisation:** after a feasible timetable is found, apply min-conflicts local search or simulated annealing to improve the soft constraints (spread exams, minimise clashes in a day) and to repair the timetable when something changes, changing only a few assignments.
- **Soft constraints as costs:** use a weighted CSP / constraint optimisation so that hard constraints must be satisfied and soft constraints are minimised.
- **Incremental and parallel solving:** keep the previous solution, re-solve only the affected variables, and run several searches with different random orderings in parallel.

Approach	Strengths	Weaknesses	Use in timetable
Plain backtracking	Simple, complete	Very slow on large problems	Only for small cases
Backtracking + MRV/LCV + forward checking / MAC	Complete, much faster, detects failure early	Still exponential in the worst case	Core solver for feasibility
Local search (min-conflicts, SA)	Scales to very large problems, good for repair	Incomplete; may get stuck	Optimisation and re-scheduling
Hybrid (propagation + search + local search)	Balances scalability, quality and flexibility	More complex to build	**Recommended**

Conclusion. The exam-timetable problem fits the CSP framework perfectly: courses are variables, time slots and halls are domains, and the rules are constraints. Backtracking search with MRV, degree and LCV heuristics together with forward checking (or arc consistency) solves moderate problems efficiently, while for large and changing real-world situations a hybrid of constraint propagation, decomposition and local search provides the necessary scalability, flexibility and quality of results.

5. Scenario: Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

ii. Analyze the computational challenges of large game trees

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

iv. Design an evaluation function and justify the factors considered

v. Recommend a strategy for balancing optimality and real-time performance

Answer 5: Strategic Game AI using Minimax and Alpha-Beta Pruning

1. Understanding the Scenario

A gaming company is building an AI opponent for a real-time strategy (RTS) game. The AI competes against human players, so every decision it makes may be answered by an opponent who is trying to defeat it. The AI must respond within strict time limits (often tens or hundreds of milliseconds per decision), must evaluate complex game states in which many units, resources and buildings exist, and faces a decision tree that is extremely large. This is an **adversarial search problem** (a two-player, zero-sum game in the basic model), for which Minimax and Alpha-Beta pruning are the classical tools.

Real RTS games are continuous, simultaneous and partially observable (fog of war). For search, the game is therefore abstracted: time is divided into decision steps, units are grouped into squads, actions are high-level commands (build, attack, expand, defend), and unseen information is estimated. With this abstraction the game can be treated as a turn-based adversarial game, which is how Minimax is applied.

i. How the Minimax Algorithm Models this Problem

Minimax models the interaction as a game tree between two players: **MAX** (our AI), who tries to maximise the value of the outcome, and **MIN** (the human opponent), who tries to minimise it. The formal elements of the game are:

- **Initial state S0:** the current game state (units, resources, positions, buildings, technology).
- **PLAYER(s):** whose turn it is, MAX or MIN.
- **ACTIONS(s):** the legal moves, such as train a unit, build a structure, move or attack with a squad, research an upgrade.
- **RESULT(s, a):** the new state after a move.
- **TERMINAL-TEST(s):** whether the game is over (a base destroyed) or the search depth/time limit has been reached.

- **UTILITY(s) / EVAL(s):** a number giving the value of the state for MAX, for example +1 win, 0 draw, −1 loss, or a heuristic score for non-terminal states.

Minimax value. The value of a node is defined recursively: MINIMAX(s) = UTILITY(s) if s is terminal; max over actions of MINIMAX(RESULT(s, a)) if it is MAX's turn; and min over actions of MINIMAX(RESULT(s, a)) if it is MIN's turn. The algorithm explores the tree depth-first to the leaves (or to a depth limit), assigns values to the leaves, and backs the values up: MAX nodes take the largest child value and MIN nodes the smallest. The move at the root with the highest backed-up value is the best move **assuming the opponent also plays optimally**.

```
MINIMAX-DECISION(state):
    return the action a in ACTIONS(state) that maximises MIN-VALUE(RESULT(state, a))

MAX-VALUE(state):
    if cutoff(state): return EVAL(state)
    v <- -infinity
    for a in ACTIONS(state): v <- max(v, MIN-VALUE(RESULT(state, a)))
    return v

MIN-VALUE(state):
    if cutoff(state): return EVAL(state)
    v <- +infinity
    for a in ACTIONS(state): v <- min(v, MAX-VALUE(RESULT(state, a)))
    return v
```

Small example. MAX has three moves A, B, C. After each, MIN has three replies with leaf values A: (3, 12, 8), B: (2, 4, 6) and C: (14, 5, 2). MIN picks the minimum in each branch: A = 3, B = 2, C = 2. MAX picks the maximum of these, so the minimax value is 3 and MAX plays move A. The result is the best guaranteed outcome against a perfect opponent, which makes Minimax a safe strategy for a competitive AI.

ii. Computational Challenges of Large Game Trees

Minimax explores the entire tree to the chosen depth. If the **branching factor** is b and the search depth is d, the time complexity is **O(b^d)** and the space is O(b·d) for depth-first exploration. This is the main source of difficulty.

- **Exponential growth:** with b = 30 and d = 6 there are about 30^6 ≈ 7×10^8 leaf nodes; with d = 10 more than 5×10^14. In an RTS game, where many units can each act, the effective branching factor is far larger than in board games such as chess (about 35).
- **Strict time limits:** the AI has to answer in a fraction of a second so that the game stays responsive. A full-depth search is impossible; the search must be cut off early.
- **Costly state evaluation and simulation:** every node requires copying or updating a complex game state and evaluating it. If one evaluation takes 10 microseconds, a tree with 10^9 nodes would take about three hours.
- **Horizon effect:** if the search stops at a fixed depth, the AI may miss an important event just beyond the horizon, such as a lost battle, or may delay the inevitable using useless moves.
- **Imperfect and simultaneous information:** fog of war and simultaneous actions mean that the true state is not known, so the model is an approximation and the minimax value is only an estimate.

- **Memory and repeated states:** the same state can be reached by different sequences of moves, causing duplicated effort unless these transpositions are detected.
- **Real-time dynamics:** the game continues while the AI thinks, so a decision that arrives late is worth less than a slightly weaker decision that arrives on time.

Branching factor b	Depth d	Leaf nodes b^d	Time at 1 million nodes/s
10	6	1,000,000	1 second
30	6	729,000,000	about 12 minutes
30	8	6.6 × 10^11	about 7.6 days
30	10	5.9 × 10^14	about 19 years

Since plain Minimax is infeasible, techniques that reduce the number of nodes examined or limit the depth are essential.

iii. How Alpha-Beta Pruning Improves Efficiency (Detailed Reasoning)

Alpha-Beta pruning computes **exactly the same minimax value and best move** as Minimax, but it avoids examining branches that cannot influence the final decision. It maintains two numbers while searching:

- **α (alpha):** the best (highest) value that MAX can already guarantee along the path from the root to the current node.
- **β (beta):** the best (lowest) value that MIN can already guarantee along that path.

Pruning rule. If at any node $\alpha \geq \beta$, the remaining children of the node are skipped (pruned). At a MIN node this means MIN has found a reply that is so good for it that MAX would never allow the game to reach this node (MAX has a better alternative elsewhere). At a MAX node it means MIN would never allow this position because MIN already has a better option higher in the tree.

```
MAX-VALUE(state, alpha, beta):
    if cutoff(state): return EVAL(state)
    v <- -infinity
    for a in ACTIONS(state):
        v <- max(v, MIN-VALUE(RESULT(state, a), alpha, beta))
        if v >= beta: return v            # prune remaining children
        alpha <- max(alpha, v)
    return v

MIN-VALUE(state, alpha, beta):
    if cutoff(state): return EVAL(state)
    v <- +infinity
    for a in ACTIONS(state):
        v <- min(v, MAX-VALUE(RESULT(state, a), alpha, beta))
        if v <= alpha: return v           # prune remaining children
        beta <- min(beta, v)
    return v
```

Worked example (same tree as before). Root = MAX with children A, B, C (MIN nodes). Start with $\alpha = -\infty$, $\beta = +\infty$.

- **Node A:** leaves 3, 12, 8. After the first leaf (3), MIN's β becomes 3; the other leaves (12, 8) do not lower it, so A = 3. At the root, $\alpha = 3$.

- **Node B:** the first leaf is 2, so β = 2. Now β (2) ≤ α (3): MAX already has a move worth 3, so B can never be better than 2 for MAX. The remaining leaves **4 and 6 are pruned**; B ≤ 2 is returned.
- **Node C:** first leaf 14 (β = 14), then 5 (β = 5), then 2 (β = 2). Here all three leaves must be examined because β stays above α until the last one. C = 2.
- **Root:** the maximum of 3, 2 and 2 is 3, so MAX plays A, the same answer as Minimax, with two leaves not examined.

Why it improves efficiency. The savings depend on **move ordering**. If the best moves are examined first, cut-offs happen early and often.

Move ordering	Nodes examined	Effect
Worst case (bad ordering)	O(b^d), same as Minimax	No pruning at all
Random / average ordering	About O(b^(3d/4))	Significant reduction
Best case (perfect ordering)	O(b^(d/2))	Search can go about **twice as deep** in the same time; effective branching factor is √b

For b = 30 and d = 8, Minimax needs about 6.6×10^11 nodes while best-case Alpha-Beta needs only about 30^4 ≈ 8×10^5, which is the difference between a week and under a second. Alpha-Beta is **lossless**: unlike heuristic cut-offs it never changes the result. Practical ordering techniques include trying captures or attacks first, using the best move from the previous (shallower) iteration, **killer moves** (moves that caused cut-offs at the same depth in sibling nodes) and the **history heuristic**.

iv. Design of an Evaluation Function and Justification of the Factors

Because the tree cannot be searched to the end of the game, the search is cut off at a certain depth or time and non-terminal states are scored by an **evaluation function** EVAL(s) that estimates the chance of winning from state s. It should be (1) consistent with the true utility on terminal states, (2) very fast to compute, because it is called millions of times, and (3) strongly correlated with the real probability of winning.

A common design is a **weighted linear function**: **EVAL(s) = w1·f1(s) + w2·f2(s) + … + wn·fn(s)**, where every feature fi is expressed as 'my value minus the opponent's value' (or a normalised ratio) so that a positive result favours the AI.

Factor (feature)	How it is measured	Why it is included
f1: Military strength	Sum over units of (unit cost × health fraction × combat effectiveness), mine minus the opponent's	Wars are decided by army strength; it captures the result of fights and the value of upgrades.
f2: Economy	Resource income rate, workers, number of expansions/bases, stockpile	A stronger economy turns into more units later; protects against short-term bias.
f3: Technology and upgrades	Tech level reached, key buildings, completed upgrades	Advanced units and upgrades give long-term advantage that is not yet visible in army size.

Factor (feature)	How it is measured	Why it is included
f4: Map control and position	Territory held, vision, control of resource points, chokepoints, high ground	Controlling space gives resources, information and tactical freedom.
f5: Base safety and defence	Health of the main buildings, defensive structures, distance of enemy squads from my base	The game is lost when the base falls, so threats to it must be weighted heavily.
f6: Tempo / initiative	Production capacity in use, idle buildings, time since the last attack	Measures how efficiently resources are being converted into strength.
f7: Information / scouting	Percentage of map seen, estimated hidden enemy strength	Reduces the risk of surprise attacks under fog of war.

Example calculation. Suppose the weights are w1 = 0.40 (military), w2 = 0.25 (economy), w3 = 0.10 (technology), w4 = 0.10 (map), w5 = 0.15 (safety). For a state where the normalised differences (−1 to +1) are f1 = +0.3, f2 = −0.2, f3 = +0.1, f4 = +0.4 and f5 = +0.5, EVAL = 0.40(0.3) + 0.25(−0.2) + 0.10(0.1) + 0.10(0.4) + 0.15(0.5) = 0.12 − 0.05 + 0.01 + 0.04 + 0.075 = **+0.195**. The positive value means the AI is slightly ahead, even though its economy is behind.

Design considerations.

- **Weights:** they can be set by expert knowledge and then tuned automatically by self-play, regression on game records, or reinforcement learning (temporal-difference learning), so the AI improves over time.
- **Normalisation and bounds:** keep EVAL within the range of terminal utilities (for example between −1 and +1) so that real wins and losses always dominate heuristic scores.
- **Speed:** use incrementally updated counters instead of recalculating everything at each node.
- **Non-linear interactions:** some factors interact (a large army is worth little with no economy), so products or phase-dependent weights (early, mid and late game) can be used.
- **Quiescence:** apply the function only to 'quiet' states (not in the middle of a battle) to avoid misleading scores.

v. Strategy for Balancing Optimality and Real-Time Performance

Perfect optimal play is impossible in real time, so the AI should aim for the best decision that can be computed inside the time limit. The recommended strategy combines several techniques:

- **Alpha-Beta pruning with iterative deepening:** search to depth 1, then 2, then 3, and so on until the time budget is used. The best move of the last completed depth is always available, so the AI can answer at any moment (anytime behaviour). Earlier iterations also provide excellent move ordering for the next one.
- **Move ordering:** killer and history heuristics and the best move from the previous iteration, which push Alpha-Beta toward its best-case O(b^(d/2)).

- **Transposition table:** a hash table of already evaluated states so that identical positions reached by different move orders are not searched again.
- **State and action abstraction:** group units into squads and use high-level commands (attack, retreat, expand) instead of individual unit moves to reduce the branching factor; search the opponent's likely replies only.
- **Depth limit and evaluation function:** use the evaluation function at the cut-off, with quiescence search to avoid the horizon effect.
- **Selective pruning:** forward pruning such as beam search (examine only the top k moves), null-move pruning or aspiration windows (search with a narrow α–β window around the previous value). These are not lossless, so they are used carefully.
- **Time management:** assign more time to critical decisions (battles) and less to routine ones, and run the search in a separate thread so that the game does not freeze.
- **Hybrid with Monte Carlo Tree Search (MCTS):** in very large, uncertain parts of the game MCTS samples random playouts and needs no handcrafted evaluation. A hybrid approach can use Alpha-Beta for tactical combat and MCTS or scripted strategies for long-term planning.
- **Opening books and scripts:** use precomputed build orders and reactions for common situations, which saves search time at the beginning of the game.
- **Safety net and difficulty control:** if the time is nearly over, play the best move found so far; difficulty levels can be created by limiting depth or adding noise to the evaluation.

Technique	Benefit	Cost / risk
Minimax alone	Optimal play within the horizon	Exponential time; impractical
Alpha-Beta pruning	Same result, far fewer nodes (up to b^(d/2))	Depends on move ordering
Iterative deepening + time limit	Always has an answer; improves ordering	Some repeated work (small)
Evaluation function + quiescence	Allows cut-off before the end of the game	Quality depends on the chosen features and weights
Transposition table	Avoids re-searching repeated states	Uses memory
Forward pruning / MCTS hybrid	Handles very large trees and uncertainty	May miss the optimal move

Conclusion. Minimax gives a sound model of competition between the AI and a human by assuming that each side chooses its best move, but its exponential cost makes it impractical for a real-time strategy game. Alpha-Beta pruning keeps the same decision quality while greatly reducing the work, especially with good move ordering. A well-designed, fast evaluation function allows the search to stop early, and combining iterative deepening, transposition tables, abstraction, selective pruning and, where suitable, Monte Carlo methods lets the AI balance near-optimal decisions with the strict time limits of real-time play.

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand